



Tabla de contenido

SCHEMAS Y MODELS EN MONGOOSE	2
Tamaños de campo	3
TIPOS DE DATOS	3
STRING:	3
NUMBER:	4
DATE:	4
BUFFER:	4
OBJECTID:	4
MIXED:	4
ARRAY:	5
Advertencia: matriz definida como tipo mixto	6
Tipos de esquema personalizados:	6
MODELOS O MODELS EN MONGOOSE	6
CONECTANDOSE A LA BASE DE DATOS	7
Cerrar la conexión	8
Llamar al comando close	8
OPERACIONES CRUD : EJEMPLO	9
CREACION DE SCHEMA Y MODEL	9
NUEVO REGISTRO	9
LISTAR	9
LISTAR POR ID	9
BORRAR UN DOCUMENTO	10



MONGOOSE

Mongoose es una herramienta de **modelado de objetos** para MongoDB y Node.js. En términos prácticos, esto significa que puedes definir tu modelo de datos en un solo lugar, en tu código.

Sí, así es. No tienes que crear un esquema en la base de datos, vincularlo a un ORM ni asignarlo a los objetos y clases de tu proyecto. Solo tienes que definir la estructura de tus datos en JSON dentro de tu proyecto.

ENLACE OFICIAL: <https://mongoosejs.com/>

Descarga desde el gestor de paquetes: <https://www.npmjs.com/package/mongoose>

SCHEMAS Y MODELS EN MONGOOSE

Un **esquema(schema)** describe fundamentalmente **la estructura de datos** de un **documento**. Este esquema define el nombre de cada elemento de datos y el tipo de datos, ya sea una cadena, un número, una fecha, un valor booleano, etc.

```
let userSchema = new mongoose.Schema({  
  name: String,  
  email: String,  
  createdAt: Date.now,  
  verified: Boolean  
});
```

En la mayoría de los casos, tendrás un esquema para cada colección dentro de la base de datos. Los esquemas son un aspecto potente de Mongoose, que también se puede ampliar con funciones auxiliares y métodos adicionales.



Tamaños de campo

Tenga en cuenta que, a diferencia de otros sistemas, no es necesario establecer el tamaño del campo. Esto puede resultar útil si necesita cambiar la cantidad de datos almacenados en un objeto concreto.

Por ejemplo, su sistema podría imponer un límite de 16 caracteres para los nombres de usuario, por lo que establecería el tamaño del campo en 16 caracteres. Más tarde, se da cuenta de que quiere cifrar los nombres de usuario, pero esto duplicará la longitud de los datos almacenados. **Si el esquema de su base de datos utiliza tamaños de campo fijos, tendrá que refactorizarlo, lo que puede llevar mucho tiempo en una base de datos grande. Con Mongoose, puede empezar a cifrar ese objeto de datos sin preocuparse por ello.**

Si almacena documentos grandes, debe tener en cuenta que MongoDB impone un tamaño máximo de documento de 16 MB. Hay formas de sortear incluso este límite, utilizando la API GridFS de MongoDB.

TIPOS DE DATOS

Hay ocho tipos de datos que se pueden configurar, de forma predeterminada, en un esquema Mongoose. También se denominan SchemaTypes (tipos de esquema); son los siguientes:

- String
- Number
- Date
- Boolean
- Buffer
- ObjectId
- Mixed
- Array

STRING: Almacena una cadena (string) en formato UTF-8



NUMBER: Este SchemaType almacena un valor numérico, con restricciones. mongoose no admite de forma nativa los tipos de datos long y double, por ejemplo, aunque **MongoDB** sí lo hace. Sin embargo, Mongoose se puede ampliar mediante complementos para admitir estos otros tipos.

DATE: Este SchemaType contiene un objeto de fecha y hora, que normalmente se devuelve desde MongoDB como un objeto **ISODate**, por ejemplo, `ISODate("2013-04-03T12:56:26.009Z")`. **BOOLEAN:** Solo tiene dos valores TRUE y FALSE

BUFFER: Este SchemaType se utiliza principalmente para almacenar información binaria, por ejemplo, imágenes almacenadas en MongoDB, El tipo Buffer es la forma más común y eficiente de manejar datos binarios directamente en un documento de MongoDB, **siempre y cuando el tamaño de los datos no supere el límite de 16 MB** por documento. Si necesitas manejar archivos más grandes, la solución recomendada es **GridFS**.

OBJECTID: Este SchemaType se utiliza para asignar un identificador único a una clave distinta de `_id`, por ejemplo, una clave externa para hacer referencia a otro documento (se verá más adelante en schemas complejos) En lugar de limitarte a especificar el tipo de ObjectId, debes especificar la versión completa `Schema.Types.ObjectId`. Por ejemplo:

```
projectSchema.add({  
  owner: mongoose.Schema.Types.ObjectId  
});
```

MIXED: Un objeto de datos mixtos puede contener cualquier tipo de datos. Se puede declarar estableciendo un objeto vacío o utilizando `Schema.Types.Mixed` Completamente calificado.

Los dos comandos siguientes harán lo mismo:

```
vardjSchema= new mongoose.Schema({  
  mixedUp: {}  
});  
  
vardjSchema= new mongoose.Schema({  
  mixedUp: Schema.Types.Mixed  
});
```



Aunque esto suena muy bien, hay una gran salvedad. Los cambios en los datos de tipo mixto no pueden ser detectados automáticamente por Mongoose, por lo que no sabe que necesita guardarlos.

Como Mongoose no puede ver automáticamente los cambios realizados en datos de tipo mixto, tienes que declarar manualmente cuándo han cambiado los datos.

Afortunadamente, Mongoose ofrece un método llamado `markModified` para hacer precisamente esto, pasándole la ruta del objeto de datos que ha cambiado.

```
dj.mixedUp = { valueone: "un nuevo valor" };
dj.markModified('mixedUp');
dj.save();
dj.mixedUp = { valueone: "a new value" };
dj.markModified('mixedUp');
dj.save();
```

ARRAY: El tipo de datos array se puede utilizar de dos maneras. En primer lugar, como una simple matriz de valores del mismo tipo de datos, tal y como se muestra en el siguiente fragmento de código:

```
var userSchema = new mongoose.Schema({
  name: String,
  emailAddresses: [String]
});
```

En segundo lugar, el tipo de datos array se puede utilizar para almacenar una colección de subdocumentos utilizando esquemas anidados. (se verá más adelante en schemas complejos) por ahora, aquí hay un ejemplo de cómo puede funcionar:

```
var emailSchema = new mongoose.Schema({
  email: String,
  verified: Boolean
});
var userSchema = new mongoose.Schema({
  name: String,
```



```
emailAddresses: [emailSchema]
});
```

Advertencia: matriz definida como tipo mixto

Si declara una matriz vacía, se tratará **como tipo mixto**, lo que significa que mongoose no podrá detectar automáticamente ningún cambio realizado en los datos. Por lo tanto, **evite estos dos tipos de declaración de matrices**, a menos que desee intencionadamente un tipo mixto.

```
var emailSchema = new mongoose.Schema({
  addresses: []
});
var emailSchema = new mongoose.Schema({
  addresses: Array
});
```

Tipos de esquema personalizados: Si sus datos requieren un tipo de datos diferente, Mongoose ofrece la opción de ampliarlo con **tipos de esquema personalizados**. El método de ampliación se gestiona mediante complementos de Mongoose, que veremos más adelante. Algunos ejemplos de ampliaciones de tipos de esquema que ya se han creado son: **long**, **double**, **RegExp** e incluso **email**.

MODELOS O MODELS EN MONGOOSE

Un modelo (Model) es una versión “compilada” del esquema. Una instancia del modelo se asignará a un documento en la base de datos. Crear una instancia de usuario basada en el esquema userSchema es una tarea de una sola línea:

```
let User = mongoose.model('User', userSchema, 'user');
```

User: corresponde al nombre a asignarle al modelo, use mayúscula inicial para denotar que es una instancia (clase)

userSchema: Es el esquema ‘compilado’ asignado al modelo



user: es un parámetro opcional para describir el modelo **NO** es obligatorio

El modelo es el que se encarga de **leer, crear, actualizar y eliminar** documentos.

CONECTANDOSE A LA BASE DE DATOS

La forma de establecer la conexión predeterminada a una base de datos MongoDB a través de Mongoose es muy sencilla, utilizando el método `mongoose.connect`.

```
let dbURI = 'mongodb://localhost/mydatabase';  
mongoose.connect(dbURI);
```

Esto abrirá una conexión Mongoose a la base de datos Mongo `mydatabase`, que se ejecuta en el servidor localhost.

Mongoose le permite pasar ciertas opciones a ambos métodos de conexión. Si se especifican, estas opciones anularán las establecidas en la cadena de conexión.

Las opciones se envían como un objeto JSON como segundo parámetro opcional a la llamada de conexión.

Por ejemplo:

```
var dbURI = 'mongodb://localhost/mydatabase';  
var dbOptions = {'user':'db_username','pass':'db_password'};  
mongoose.connect(dbURI, dbOptions);
```

Las opciones que puede utilizar de esta manera son:

- **user y pass:** nombre de usuario y contraseña para la base de datos, si son necesarios y no se especifican en la cadena de conexión.
- **db:** se refiere a las opciones de base de datos disponibles en el controlador nativo Node MongoDB.
- **server:** se refiere a las opciones de servidor disponibles en el controlador nativo Node MongoDB Native.
- **replset:** esta opción le permite especificar un ReplicaSet. Los detalles de los métodos ReplicaSet, permite tener una base de datos principal donde se realizan todas las escrituras, y varias bases de datos secundarias. Si la base de datos principal



falla, una de las bases de datos secundarias se convierte automáticamente en la nueva principal.

Cerrar la conexión

Como ya hemos visto, la mejor práctica general es abrir la conexión al iniciar la aplicación y mantenerla abierta. Sin embargo, hay ocasiones en las que querrás cerrar la conexión. Por ejemplo, si tu aplicación se está cerrando o reiniciando, la conexión a la base de datos debe cerrarse manualmente, o si estás ejecutando un script de un solo uso en lugar de una aplicación persistente.

Llamar al comando close

Cada conexión Mongoose tiene un método `close()` que toma una función callback opcional. Si estás utilizando la conexión predeterminada, lo llamas como lo hace el siguiente fragmento de código:

```
mongoose.connection.close(function () {  
  console.log('Mongoose default connection closed');  
});
```




OPERACIONES CRUD : EJEMPLO

Ejemplo para una COLLECTION denominada ARTICULO que almacenará la información de las entradas o artículos publicados en un blog.

CREACION DE SCHEMA Y MODEL

```
1 import { Schema, model } from "mongoose";
2
3 const ArtículoSchema = Schema({
4   titulo: {
5     type: String,
6     required: true,
7   },
8   contenido: {
9     type: String,
10    required: true,
11  },
12  fecha: {
13    type: Date,
14    default: Date.now,
15  },
16  imagen: {
17    type: String,
18    default: "default.png",
19  },
20 });
21 // crea el objeto con nombre del objeto, esquema correspondiente y nombre alternativo
22 export default model("Articulo", ArtículoSchema);
```

NUEVO REGISTRO

```
import Artículo from "../models/Articulo.js";
const articulo = new Artículo(parametros);
const articuloGuardado = await articulo.save();
```

LISTAR

```
import Artículo from "../models/Articulo.js";
let consulta = Artículo.find({}).sort({ fecha: -1 });
```

LISTAR POR ID

```
import Artículo from "../models/Articulo.js";
const articulo = await Artículo.findById(id);
```



BORRAR UN DOCUMENTO

```
import Artículo from "../models/Articulo.js";

const articuloBorrado = await Artículo.findOneAndDelete({ _id: articuloId
});
```

EDITAR UN DOCUMENTO

```
import Artículo from "../models/Articulo.js";

const articuloActualizado = await Artículo.findOneAndUpdate({ _id:
articuloId }, parametros, { new: true });
```